

Estructuras de Datos y Algoritmos

Repaso de programación y Python

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Propósito

Recuperar las herramientas de programación, Python y terminal que utilizaremos para diseñar, ejecutar y probar tipos de datos abstractos.

Programación

Datos, control, funciones y estado.

Python

Objetos, mutabilidad, errores y módulos.

Terminal

Navegar, ejecutar programas y correr pruebas.

Cómo leer un programa

Pregunta	Qué debemos localizar
Datos	Valores que entran al programa y objetos que se crean.
Control	Orden de ejecución, decisiones y repeticiones.
Estado	Información que puede cambiar durante la ejecución.
Resultado	Valores devueltos, impresos o almacenados.

Estrategia

Realizar una traza significa seguir el programa paso a paso y registrar cómo cambian los objetos involucrados.

Nombres, objetos y tipos en Python

```
attempts = 3  
student = "Ana"  
values = [2, 4]  
approved = True
```

Lectura

`attempts`, `student` y `values` son nombres asociados con objetos.

- 3 tiene tipo `int`.
- Ana tiene tipo `str`.
- [2, 4] tiene tipo `list`.
- `approved` tiene tipo `bool`.

Asignación y evaluación de expresiones

```
price = 12.5  
pieces = 4  
total = price * pieces
```

Para ejecutar la tercera línea, Python:

- 1 busca los objetos asociados con `price` y `pieces`;
- 2 evalúa la expresión `12.5 * 4`;
- 3 obtiene el resultado `50.0`;
- 4 asocia el nombre `total` con ese resultado.

Orden de una asignación

Primero se evalúa el lado derecho; después se realiza la asignación indicada en el lado izquierdo.

Objetos inmutables y mutables

Tipo inmutable: int

```
a = 1
b = a
b += 3
print(a)      # 1
print(b)      # 4
```

El objeto 1 no puede modificarse. `b += 3` obtiene 4 y asocia `b` con ese resultado.

Tipo mutable: list

```
a = [1, 2]
b = a
b += [3]
print(a)      # [1, 2, 3]
print(b)      # [1, 2, 3]
```

La lista sí puede modificarse. `b += [3]` extiende el objeto compartido por `a` y `b`.

Comparación

En ambos casos, `b = a` asocia dos nombres con un mismo objeto. La diferencia aparece cuando `+=` actúa sobre un tipo inmutable o mutable.

Objetos inmutables y mutables

Inmutables

Su valor interno no puede cambiar.

- Ejemplos: `int`, `float`, `bool`, `str`.
- Una operación produce otro resultado.
- El nombre se asocia con ese resultado; los otros nombres no cambian.

Mutable

Su estado interno puede cambiar.

- Ejemplos: `list`, `dict` y `set`.
- Una operación puede modificar el mismo objeto.
- Todos los nombres asociados con él observan el cambio.

Idea esencial

La mutabilidad es una propiedad del **objeto**, no del nombre. La asignación `b = a` comparte el objeto; la operación posterior determina si el objeto cambia o si `b` se asocia con otro resultado.

Colecciones integradas de Python

Tipo	Ejemplo	Orden	Cambio
Lista	[2, 4, 6]	por posición	mutable
Tupla	(2, 4, 6)	por posición	inmutable
Diccionario	{'Ana': 8}	por inserción	mutable
Conjunto	{2, 4, 6}	sin posición	mutable

Decisiones: if, elif y else

```
def status(score):  
    if score >= 90:  
        return "sobresaliente"  
    elif score >= 60:  
        return "aprobado"  
    else:  
        return "no aprobado"
```

Condición

Una expresión que produce True o False.

Caso frontera

¿Qué debe regresar la función para 59, 60, 89 y 90?

La indentación determina qué instrucciones pertenecen a cada rama.

Operadores lógicos en una condición

```
if enrolled and (on_time or authorized):  
    accept_submission()  
  
if not file_exists:  
    report_error()
```

Operador	El resultado es verdadero cuando...
and	ambas condiciones son verdaderas.
or	al menos una condición es verdadera.
not	la condición original es falsa.

Precedencia

Python evalúa primero not, después and y finalmente or. Los paréntesis hacen explícita la intención.

Repeticiones: for y while

for: recorrer una colección

```
total = 0
for value in [3, 1, 4]:
    total += value
```

Paso	value	total
inicial	–	0
1	3	3
2	1	4
3	4	8

while: repetir bajo una condición

```
count = 3
while count > 0:
    print(count)
    count -= 1
```

Evaluación	count	count > 0
1	3	True
2	2	True
3	1	True
4	0	False

Diferencia clave

for obtiene el siguiente elemento del iterable. En while, el cuerpo debe modificar el estado que controla la condición; de lo contrario, el ciclo puede no terminar.

Funciones

```
def average(values):  
    return sum(values) / len(values)
```

Elemento	Interpretación
average	Nombre de la operación.
values	Parámetro que recibe la entrada.
return	Entrega el resultado al código que llamó la función.
len(values) > 0	Condición necesaria para esta implementación.

Idea clave

Para usar una función necesitamos conocer su interfaz y comportamiento, no cada paso de su implementación.

Alcance, estado y efectos secundarios

Produce un nuevo objeto

```
def increment(number):  
    return number + 1
```

```
x = 4  
y = increment(x)
```

Modifica un objeto recibido

```
def append_zero(values):  
    values.append(0)
```

```
data = [1, 2]  
append_zero(data)
```

Efecto secundario

Cambio observable fuera de la función, como modificar un objeto mutable, escribir un archivo o imprimir en pantalla.

¿Cuál de las llamadas cambia el estado de un objeto existente?

Mutabilidad y aliasing no son lo mismo

Mutabilidad

Es una **propiedad de un objeto**: indica si su estado interno puede cambiar sin sustituirlo por otro objeto.

Aliasing

Es una **relación entre nombres**: ocurre cuando dos o más nombres están asociados con el mismo objeto.

Situación	Código	Resultado
Aliasing e inmutabilidad	<code>a = 1; b = a; b += 1</code>	<code>a == 1, b == 2</code>
Mutabilidad sin aliasing	<code>a = [1]; b = [1]; b.append(2)</code>	<code>a == [1]</code>
Mutabilidad con aliasing	<code>a = [1]; b = a; b.append(2)</code>	<code>a == [1, 2]</code>

Idea central

El aliasing no implica mutabilidad y la mutabilidad no implica aliasing. Los cambios compartidos aparecen cuando varios nombres están asociados con un mismo objeto mutable.

Igualdad, identidad y copia

```
a = [1, 2]
b = a
c = a.copy()

print(a == c)    # mismo contenido
print(a is c)    # objetos distintos
print(a is b)    # el mismo objeto
```

Igualdad: ==

Compara el contenido o valor definido por el tipo.

Identidad: is

Comprueba si ambos nombres se refieren al mismo objeto.

`copy()` crea otra lista, aunque sus elementos internos todavía pueden compartirse.

Excepciones: hacer explícito un error

```
def percentage(part, total):  
    if total <= 0:  
        raise ValueError("total must be positive")  
    return 100 * part / total
```

- Una excepción interrumpe el flujo normal de una operación.
- Su tipo comunica la clase de problema.
- Su mensaje aporta contexto para diagnosticarlo.

Conexión con el diseño

Si una entrada no es válida, el comportamiento esperado también debe formar parte de la especificación.

Cómo leer un traceback

```
Traceback (most recent call last):  
  File "main.py", line 4, in <module>  
    print(average([]))  
  File "course_stats.py", line 2, in average  
    return sum(values) / len(values)  
ZeroDivisionError: division by zero
```

- 1 Comienza por la **última línea**: tipo y mensaje del error.
- 2 Localiza la **primera línea de tu código** que debes investigar.
- 3 Sigue la cadena de llamadas para reconstruir cómo se llegó al error.

Pregunta

¿El problema está en la división o en permitir una lista vacía?

Scripts, módulos e importaciones

course_stats.py

```
def average(values):  
    return sum(values) / len(values)
```

main.py

```
from course_stats import average  
  
def main():  
    print(average([8, 9, 10]))  
  
if __name__ == "__main__":  
    main()
```

- Un archivo .py puede utilizarse como módulo o como script.
- Importar permite reutilizar funciones y separar responsabilidades.
- El bloque `__main__` se ejecuta solamente al correr el archivo directamente.

Clases y objetos: primer vistazo

```
class Student:
    def __init__(self, name):
        self.name = name
        self.score = 0

    def add_points(self, points):
        self.score += points

ana = Student("Ana")
ana.add_points(5)
```

- **Clase:** definición que agrupa datos y comportamiento.
- **Objeto:** instancia concreta de una clase.
- **Atributos:** datos que forman su estado.
- **Métodos:** operaciones que puede realizar.
- **self:** objeto sobre el que actúa el método.

Distinción importante

Una clase es un mecanismo de Python. Puede utilizarse para implementar un TDA, pero clase y TDA no son sinónimos.

Terminal y shell en distintos sistemas

Terminal

Aplicación o interfaz desde la que escribimos comandos y observamos sus resultados.

Shell

Programa que interpreta los comandos y solicita al sistema que los ejecute. Bash es uno de varios shells posibles.

Sistema	Aplicaciones de terminal	Ejemplos de shells
Windows	Windows Terminal, terminal de VS Code	PowerShell, cmd, Bash con Git Bash o WSL
Linux	GNOME Terminal, Konsole, terminal de VS Code	Bash, Zsh, Fish
macOS	Terminal, iTerm2, terminal de VS Code	Zsh, Bash

Idea clave

El sistema operativo no obliga a utilizar un único shell. El shell conserva un directorio de trabajo desde el cual interpreta las rutas relativas y ejecuta nuestros programas.

Nota: Anaconda Prompt abre cmd preconfigurado para Conda; no es un shell adicional.

Rutas y navegación en Windows y Linux

Acción	Windows: cmd	Linux: Bash
Mostrar la carpeta actual	<code>cd</code>	<code>pwd</code>
Listar el contenido	<code>dir</code>	<code>ls</code>
Entrar a una carpeta	<code>cd practica_0</code>	<code>cd practica_0</code>
Subir un nivel	<code>cd ..</code>	<code>cd ..</code>
Crear una carpeta	<code>mkdir resultados</code>	<code>mkdir resultados</code>
Ejecutar un script	<code>python archivo.py</code>	<code>python3 archivo.py</code>

Ruta relativa

Se interpreta desde el directorio actual. Ejemplo: `practica_0`.

Los ejemplos de Windows suponen cmd o Anaconda Prompt. PowerShell también permite `pwd`, `ls`, `cd` y `mkdir`. Con el entorno Conda activo, usaremos `python archivo.py` también en Linux.

Ruta absoluta

Identifica una ubicación completa dentro del sistema de archivos.

Comprobación 1: recorrer y actualizar un diccionario

```
def update(scores, bonus):  
    for name in scores:  
        if scores[name] >= 60:  
            scores[name] += bonus  
    return scores  
  
grades = {"Ana": 55, "Luis": 70}  
result = update(grades, 5)  
  
print(grades)  
print(result is grades)
```

- 1 ¿Qué imprime cada print?
- 2 ¿Qué valores cambian durante la ejecución?
- 3 ¿Por qué `result is grades` produce `True` y cómo evitarías modificar `grades`?

Comprobación 2: seleccionar y acumular

```
def analyze(numbers):  
    selected = []  
    total = 0  
    for number in numbers:  
        if number % 2 == 0:  
            selected.append(number)  
            total += number  
    return selected, total  
  
data = [1, 2, 3, 4, 5]  
evens, total = analyze(data)  
  
print(evens)  
print(total)  
print(data)
```

- 1 ¿Qué imprime cada print?
- 2 ¿La función modifica la lista data?
- 3 ¿Qué valores toma total durante la ejecución?

Comprobación 3: ciclos anidados

```
def row_totals(matrix):  
    result = []  
    for row in matrix:  
        subtotal = 0  
        for value in row:  
            if value > 0:  
                subtotal += value  
        result.append(subtotal)  
    return result  
  
data = [[2, -1, 3], [0, 4, -2]]  
totals = row_totals(data)  
  
print(totals)  
print(data)
```

- 1 ¿Qué imprime cada print?
- 2 ¿Cuántas veces se ejecuta el ciclo interior?
- 3 ¿Qué ocurriría si una fila estuviera vacía?

Ejercicio: de for a while

En la slide anterior, el ciclo exterior era `for row in matrix`. Esta versión ya contiene la estructura `while`: completa sus tres espacios con la condición para continuar, la fila correspondiente a `index` y la actualización del índice. El resultado debe ser el mismo y `matrix` no debe modificarse.

```
def row_totals(matrix):  
    result = []  
    index = 0  
    while _____:  
        row = _____  
        subtotal = 0  
        for value in row:  
            if value > 0:  
                subtotal += value  
        result.append(subtotal)  
        _____  
    return result
```

Casos para verificar

`[[2, -1, 3], [0, 4, -2]], [[]], [1, -1]]` y `[]`.